Aufgabe 1: (10 Punkte)

Beschreiben Sie in eigenen Worten was ist unter dem Begriff „Software-Engineering“ zu verstehen ist und welche Ziele damit verfolgt werden (Nennen Sie mindestens 5)?

*Software Engineering beschäftigt sich mit der **systematischen Erstellung** von Software. Software Engineering verwendet **Prinzipien, Methoden und Werkzeuge** um ein Software Produkt im gewünschten **Funktionsumfang, in Qualität, im gegebenen Budgetrahmen und zu einem geplanten Termin** liefern zu können.*

Aufgabe 2: (10 Punkte)

Lesen Sie folgende Aussagen genau durch und bewerten Sie sie mit einem Kreuz als richtig oder falsch

Aussage	richtig	falsch
Die Erstellung eines Prototyps dient primär der Verringerung der Entwicklungszeit eines Systems?		X
Es ist nicht sinnvoll, mehr als eine Methode zur Modellierung eines Systems zu verwenden, da mit allen (in der Vorlesung behandelten) Methoden die gleichen Aspekte eines Systems betrachtet werden		X
In einem Blockdiagramm kann ein Agent sowohl mit Agenten als auch mit Storage Elementen in Beziehung gebracht werden	X	
Bei einem Software Architekt sind profunde Kenntnisse moderner Modellierungstechniken weit wichtiger als Kommunikationsfähigkeit		X
Anwendungsfalldiagramme (Use Case Diagram) wird zu der Gruppe der UML Verhaltensdiagramme gezählt	X	

Aufgabe 3: (10 Punkte)

Wozu dient ein Sequenzdiagram und für was wird dessen Sequentialisierung verwendet?

*Das **Sequenzdiagramm ist ein Verhaltensdiagramm** und zeigt die Interaktionssicht auf die dynamischen Aspekte eines Systems. Ein Sequenzdiagramm ist eine grafische Darstellung einer Interaktion und beschreibt den Austausch von Nachrichten zwischen Ausprägungen mittels Lebenslinien.*

Die Sequentialisierung dient der Darstellung eines Funktionsablaufs. Der Sinn ist es, eine Reihenfolge zu finden, in der die Aktionen hintereinander ausgeführt werden können, so dass jede Aktion, die das Ergebnis einer anderen benötigt, erst ausgeführt wird, wenn dieses Ergebnis vorliegt.

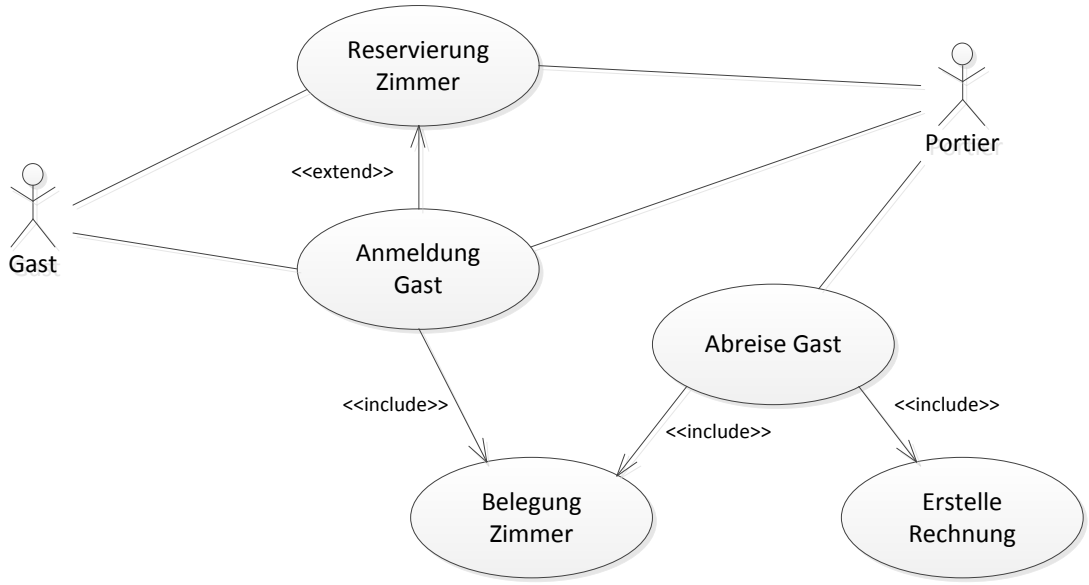
Aufgabe 4: (40 Punkte)

Gegeben ist folgende Systemspezifikation eines **Hotel Raumbuchungssystems**

Ein Hotelbuchungssystem soll erstellt werden mit folgendem funktionalen Umfang.

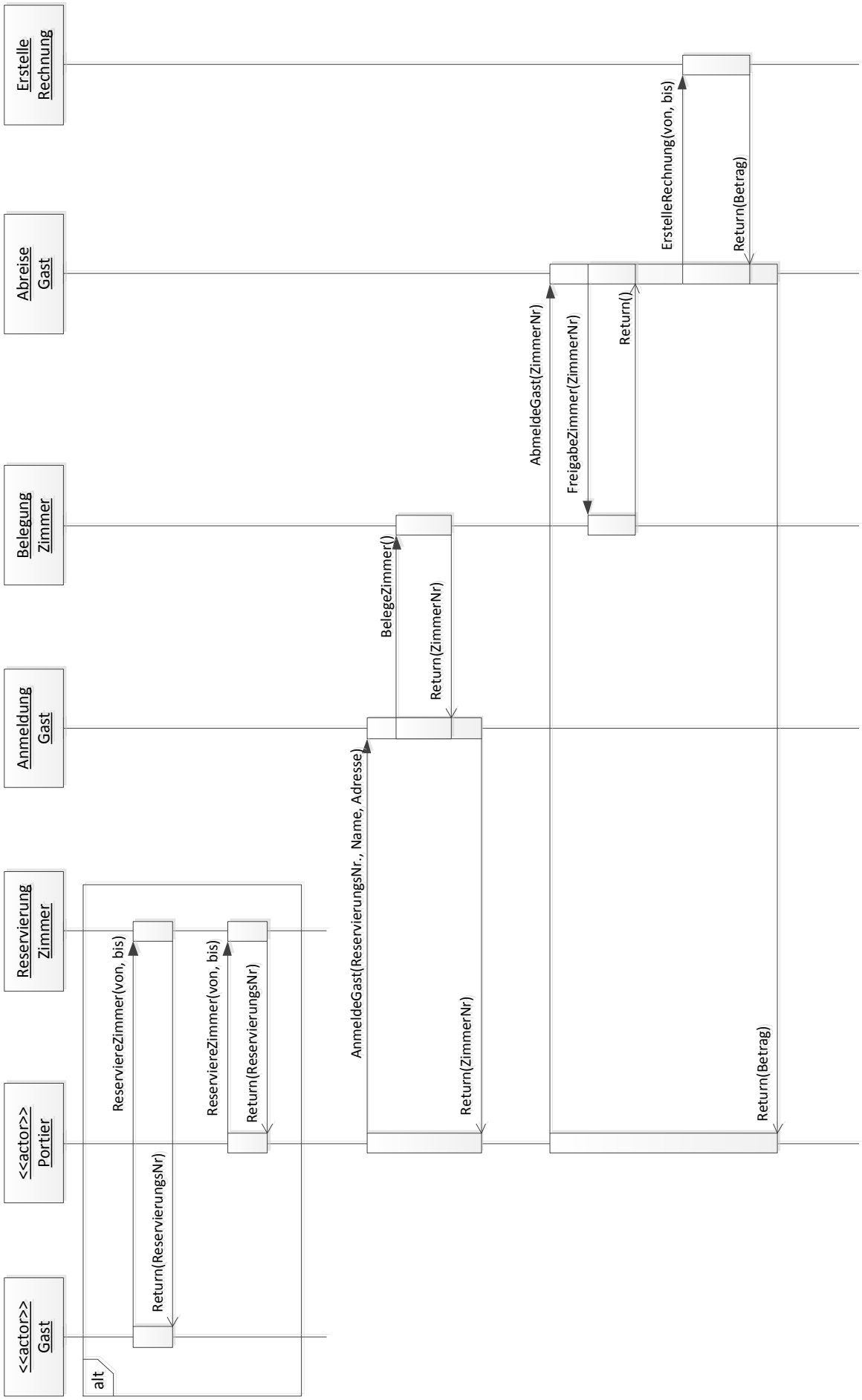
- Ein Gast soll eine Zimmerreservierung durchführen können. Alternativ kann diese vom Portier für den Kunden (die vorher via Fax oder Telefon eingegangen ist) durchgeführt werden. Dazu soll die Funktion ‚Reservierung Zimmer' genutzt werden.
- Wenn ein Kunde im Hotel ankommt wird die Anmeldung (Funktion: ‚Anmeldung Gast') vom Portier durchgeführt. Dabei kann auf eine vorher durchgeführte Reservierung zugegriffen werden. Alternativ kann eine Anmeldung auch ohne vorherige Reservierung erfolgen.
- Eine durchgeführte Anmeldung eines Gastes führt automatisch ein Zimmer Belegung durch.
- Der Portier soll bei der Abreise eines Gastes soll automatisch die Belegung des Zimmers freigeben und eine Rechnung erstellt werden. Dazu benutzt der Portier die Funktion ‚Abreise Gast'

a.) Erstellen Sie bitte basierend auf folgender Spezifikation ein Anwendungsfall (Use Case) Diagramm, mit Nutzung von <<include>> und <<exclude>> Beziehungen. Halten Sie sich dabei bitte genau an die unten angegeben Beschreibung, auch wenn Ihre Vorstellung umfangreicher sein kann.



b.) Erstellen Sie basierend auf o. g. Systembeschreibung ein Sequenzdiagramm:

- Nutzen sie Ihre in (a) erstellten Use Case Funktionen (z. B. Reservierung Zimmer, Anmeldung Gast, …) zur Notation der Aktoren und Klassen/Objekte
- Zur Vereinfachung können sie die Option einer Anmeldung ohne Reservierung weglassen und nur die Anmeldung des Gastes mit einer gültigen Reservierung darzustellen
- Zur besseren Lesbarkeit ergänzen sie die Nachrichten mit folgenden Methoden-Vorlagen:
    i. ReserviereZimmer(von,bis) | Return(ReservierungsNr)
    ii. AnmeldeGast(ReservierungsNr, Name, Adresse) | Return(ZimmerNr)
    iii. BelegeZimmer() | Return(ZimmerNr)
    iv. AbmeldeGast(ZimmerNr) | Return(Betrag)
    v. FreigabeZimmer(ZimmerNr) | Return()
    vi. ErstelleRechnung(von, bis) | Return(Betrag)
- Zur Vereinfachung nutzen Sie bitte den Vordruck auf Seite 5


<<actor>>
Gast
<<actor>>
Portier
Reservierung
Zimmer
Anmeldung
Gast
Belegung
Zimmer
Abreise
Gast
Erstelle
Rechnung
alt
ReserviereZimmer(von, bis)
Return(ReservierungsNr)
ReserviereZimmer(von, bis)
Return(ReservierungsNr)
AnmeldeGast(ReservierungsNr., Name, Adresse)
BelegeZimmer()
Return(ZimmerNr)
Return(ZimmerNr)
AbmeldeGast(ZimmerNr)
FreigabeZimmer(ZimmerNr)
Return()
ErstelleRechnung(von, bis)
Return(Betrag)
Return(Betrag)

Aufgabe 5: (10 Punkte)

Ein Prinzip der Software Architektur ist ‚Koppelung und Kohäsion'.

a) Was versteht man unter den beiden Begriffen?

Kopplung bezeichnet Abhängigkeiten zwischen Komponenten und Teilsystemen. Kopplung der verschiedenen Komponenten sollte möglichst gering sein

Kohäsion bezeichnet den Zusammenhalt von Entitäten innerhalb einer Komponente.

b) Auf welche Systemeigenschaften wirkt sich die Kopplung besonders positiv aus:

Je unabhängiger die einzelnen Komponenten voneinander sind, desto einfacher lässt sich ein System: verstehen, testen, erweitern und ändern

Aufgabe 6: (20 Punkte)

Eine der verbreitetsten agilen Entwicklungsmodelle ist das Extreme Programming (XP) welches häufige und automatisierte Tests durch Test Driven Development (TDD) beinhaltet. In der Industrie wird häufig der im ersten Augenblick erhöhte Testaufwand für eine 100%ige Testabdeckung kontrovers diskutiert. Nennen Sie mindesten 2 Argumente für die Anwendung von TDD sowie mindestens 2 Gegenargrumente und führen Sie diese genauer aus.

Vorteile:

- *Geringere Fehlerquote dadurch die Fehler werden frühzeitig im Entwicklungsprozess erkannt*
- *Nach jeder Änderung kann verifiziert werden ob das System noch richtig funktioniert*

Nachteile:

- *Entwicklungsgeschwindigkeit erscheint reduziert bzw. langsam, da vor der eigentlichen Funktionsentwicklung zuerst die Testfunktion entwickelt werden muss*
- *Erfordert Erfahrung um aussagekräftige Testdaten zu definieren*